Задания для закрепления материала по теме:
«Комментирование кода в Android Studio: Java и XML»

Часть 1: Основные принципы

Правило 90/10 для комментариев:

- Хороший комментарий объясняет ПОЧЕМУ (90% случаев)
- Плохой комментарий дублирует ЧТО делает код (10% случаев)

Задание: Отметьте ✔, какие цели являются правильными для комментариев.

- ✔ Объяснить неочевидную бизнес-логику
- ✔ Объяснить, почему выбран именно такой алгоритм
- ✔ Описать сложную работу с View в Android
- Перевести названия переменных с английского
- ✔ Объяснить назначение сложного XML-макета
- ✔ Указать на временное решение (TODO, FIXME)
- Закомментировать старый код "на всякий случай"

Часть 2: Комментирование Java-кода в Android

Типы комментариев в Java:

Задание 1: Сопоставьте тип комментария с его описанием.

Тип	Синтаксис	Назначение
Строчный	`// ...`	с
Блочный	`/* ... */`	в
JavaDoc	`/** ... */`	а

Варианты:

- А. Документирование публичных классов, методов

- В. Временное отключение кода, многострочные пояснения

- С. Краткие пояснения к одной строке

Задание 2: Анализ Java-комментариев

Плохой пример:

```
java
// Проверяем пермишен
if (ContextCompat.checkSelfPermission(this, Manifest.permission.CAMERA) == PackageMan
ager.PERMISSION_GRANTED) {
    // Запускаем камеру
    startCamera();
}
```

Почему это плохо? – комментарий пересказывает, т о что видно по коду

Хороший пример:

```java
// Явно проверяем разрешение для камеры, так как на Android 6.0+
// требуется runtime permission даже если оно объявлено в манифесте
if (ContextCompat.checkSelfPermission(this, Manifest.permission.CAMERA) == PackageMan
ager.PERMISSION_GRANTED) {
    startCamera();
} else {
    // Запрашиваем разрешение - система покажет диалог
    ActivityCompat.requestPermissions(this, new String[]{Manifest.permission.CAMERA},
CAMERA_REQUEST_CODE);
}
```

Задание 3: Напишите JavaDoc для метода

```java
/**
 * форматирует время в миллисекундах в строку с датой
 *
 * @param timestamp вреям в миллисекундах с начала
 * @return готовая строка даты
 */
public static String formatDate(Context context, long timestamp) {
    DateFormat dateFormat = android.text.format.DateFormat.getDateFormat(context);
    return dateFormat.format(new Date(timestamp));
}
```

Часть 3: Комментирование XML-разметки в Android

Особенности XML-комментариев:

- Используется только блочный синтаксис: `<!-- Комментарий -->`

- Не поддерживают JavaDoc-форматирование

- Важны для объяснения сложных layout-структур

Задание 4: Проанализируйте XML-комментарии

Плохой пример:

```xml
```

```xml
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical">

    <!-- Текст -->
    <TextView
        android:id="@+id/title"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />

    <!-- Кнопка -->
    <Button
        android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />
</LinearLayout>
```

Почему это плохо? – коммент дублирует название виджета (не объясняет назначение и зачем используется)

Хороший пример:

```xml
<!-- Контейнер для элементов профиля пользователя -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="16dp">

    <!-- Заголовок профиля - отображает имя пользователя -->
    <TextView
        android:id="@+id/profile_title"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textSize="24sp"
        android:textStyle="bold" />

    <!-- Кнопка редактирования профиля -->
    <!-- VISIBLE/GONE в зависимости от прав текущего пользователя -->
    <Button
        android:id="@+id/edit_profile_button"
        android:layout_width="wrap_content"
```

```
        android:layout_height="wrap_content"
        android:text="@string/edit_profile" />
</LinearLayout>
```

Задание 5: Напишите комментарии для сложного макета

```
xml
<ConstraintLayout>
<!-- аватарка пользователя -->
    <ImageView
        android:id="@+id/avatar"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"/>
<!-- ник (привязан к аватарке и отображается справа) -->
    <TextView
        android:id="@+id/username"
        app:layout_constraintStart_toEndOf="@id/avatar"
        app:layout_constraintTop_toTopOf="@id/avatar"/>
<!-- вермя последнего посещения (отображается под ником) -->
    <TextView
        android:id="@+id/last_seen"
        app:layout_constraintStart_toEndOf="@id/avatar"
        app:layout_constraintTop_toBottomOf="@id/username"/>
<!-- индикатор загрузки (отображается в нижнем правом углу) -->
    <ProgressBar
        android:id="@+id/loading_indicator"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"/>
</ConstraintLayout>
```

Часть 4: Проверка качества комментариев

Перед отправкой кода в репозиторий проверьте:

- Комментарии объясняют ПОЧЕМУ, а не ЧТО
- Нет закомментированного "мертвого" кода
- JavaDoc присутствует для публичных методов
- Сложные участки кода имеют пояснения
- XML-макеты с нетривиальной структурой прокомментированы